

C# / .NET Learning Roadmap

Topic Notes — Section 17: Clean Architecture / Project Structuring

Topics: 17.1 through 17.3

Date: 29 June 2026

Section 17 — Clean Architecture / Project Structuring

This section covers all three roadmap topics: 17.1 (Layered architecture -- Domain, Application, Infrastructure, API), 17.2 (Separation of concerns across layers), 17.3 (Repository and Unit of Work pattern in practice). This is the final section of the Tier 1 roadmap.

17.1. Layered Architecture (Domain, Application, Infrastructure, API)

Builds on: 9.1 (SOLID -- layered architecture applies SRP and DIP), 9.2 (repository -- lives in Infrastructure, abstracted in Application), 12.1 (.NET CLI -- each layer is a separate project).

Why layered architecture?

Without layered architecture:

- Controllers call DbContext directly
- Business logic scattered across controllers
- Testing requires a real database
- Changing EF Core means editing controllers

With layered architecture:

- Each layer has one clear responsibility
- Business logic has no knowledge of HTTP or EF Core
- Testing uses fakes -- no real database needed
- Swap EF Core for Dapper: only Infrastructure changes

The four layers

```
+-----+
|           API Layer           | <- outermost
| Controllers, Middleware, Filters, DTOs |
+-----+
|           Application Layer    |
| Use cases, Services, Interfaces, |
| Validators, CQRS Handlers      |
+-----+
|           Infrastructure Layer  |
| EF Core, Repositories, Email,  |
| External APIs, File Storage    |
+-----+
|           Domain Layer        | <- innermost
| Entities, Value Objects, Enums, |
| Domain Events, Business Rules  |
+-----+
```

Dependency rule: outer layers depend on inner.
Inner layers NEVER depend on outer layers.

Domain layer -- no dependencies

```
// MyApp.Domain.csproj -- ZERO project references

public class Order
```

```

{
    public int Id { get; private set; }
    public string CustomerName { get; private set; }
        = string.Empty;
    public decimal Amount { get; private set; }
    public OrderStatus Status { get; private set; }
        = OrderStatus.Pending;

    private readonly List<OrderItem> _items = new();
    public IReadOnlyList<OrderItem> Items
        => _items.AsReadOnly();

    // Business rules live in the domain:
    public void AddItem(OrderItem item)
    {
        if (Status != OrderStatus.Pending)
            throw new DomainException(
                "Cannot add items to non-pending order");
        _items.Add(item);
        Amount = _items
            .Sum(i => i.Quantity * i.UnitPrice);
    }

    public void Cancel()
    {
        if (Status == OrderStatus.Shipped)
            throw new DomainException(
                "Cannot cancel a shipped order");
        Status = OrderStatus.Cancelled;
    }
}

```

Application layer -- depends on Domain only

```

// MyApp.Application.csproj
// References: Domain only

// Interfaces defined here, implemented in Infra:
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(
        int id, CancellationTokens ct);
    Task<Order> CreateAsync(
        Order order, CancellationTokens ct);
    Task UpdateAsync(
        Order order, CancellationTokens ct);
}

public interface IEmailService
{
    Task SendOrderConfirmationAsync(
        Order order, CancellationTokens ct);
}

// Service orchestrates the use case:

```

```

public class OrderService
{
    private readonly IOrderRepository _repo;
    private readonly IEmailService _email;

    public OrderService(
        IOrderRepository repo,
        IEmailService email)
    { _repo = repo; _email = email; }

    public async Task<Order> CreateOrderAsync(
        CreateOrderCommand cmd,
        CancellationToken ct)
    {
        var order = new Order();
        foreach (var item in cmd.Items)
            order.AddItem(item.ToEntity());

        var saved = await _repo
            .CreateAsync(order, ct);
        await _email
            .SendOrderConfirmationAsync(saved, ct);
        return saved;
    }
}

```

Infrastructure layer -- implements interfaces

```

// MyApp.Infrastructure.csproj
// References: Application + Domain

public class SqlOrderRepository : IOrderRepository
{
    private readonly AppDbContext _context;

    public SqlOrderRepository(AppDbContext ctx)
        => _context = ctx;

    public async Task<Order?> GetByIdAsync(
        int id, CancellationToken ct)
        => await _context.Orders
            .Include(o => o.Items)
            .FirstOrDefaultAsync(
                o => o.Id == id, ct);

    public async Task<Order> CreateAsync(
        Order order, CancellationToken ct)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync(ct);
        return order;
    }

    public async Task UpdateAsync(
        Order order, CancellationToken ct)

```

```

    {
        _context.Orders.Update(order);
        await _context.SaveChangesAsync(ct);
    }
}

```

API layer -- depends on Application

```

// MyApp.Api.csproj
// References: Application + Infrastructure
//   (Infrastructure only for DI wiring)

[ApiController]
[Route("api/orders")]
public class OrdersController : ControllerBase
{
    private readonly OrderService _service;

    public OrdersController(OrderService service)
        => _service = service;

    [HttpPost]
    public async Task<IActionResult> Create(
        CreateOrderDto dto, CancellationToken ct)
    {
        var order = await _service
            .CreateOrderAsync(dto.ToCommand(), ct);
        return CreatedAtAction(
            nameof(GetById),
            new { id = order.Id },
            order.ToDto());
    }
}

// Program.cs -- DI wiring:
builder.Services.AddDbContext<AppDbContext>(
    o => o.UseSqlServer(connStr));
builder.Services
    .AddScoped<IOrderRepository,
        SqlOrderRepository>();
builder.Services
    .AddScoped<IEmailService,
        SmtEmailService>();
builder.Services.AddScoped<OrderService>();

```

Solution structure

```

MyApp.sln
■■■ src/
■   ■■■ MyApp.Domain/
■   ■   ■■■ Entities/Order.cs
■   ■   ■■■ ValueObjects/Address.cs
■   ■   ■■■ Enums/OrderStatus.cs
■   ■   ■■■ Exceptions/DomainException.cs
■   ■   ■■■ MyApp.Domain.csproj (no refs)

```

```

■ ■■■ MyApp.Application/
■ ■ ■■■ Interfaces/IOrderRepository.cs
■ ■ ■■■ Services/OrderService.cs
■ ■ ■■■ Commands/CreateOrderCommand.cs
■ ■ ■■■ DTOs/OrderDto.cs
■ ■ ■■■ Validators/CreateOrderValidator.cs
■ ■ ■■■ MyApp.Application.csproj
■ ■ (references: Domain)
■ ■■■ MyApp.Infrastructure/
■ ■ ■■■ Persistence/AppDbContext.cs
■ ■ ■■■ Repositories/SqlOrderRepository.cs
■ ■ ■■■ Email/SmtEmailService.cs
■ ■ ■■■ MyApp.Infrastructure.csproj
■ ■ (references: Application, Domain)
■ ■■■ MyApp.Api/
■ ■ ■■■ Controllers/OrdersController.cs
■ ■ ■■■ Middleware/ExceptionMiddleware.cs
■ ■ ■■■ Program.cs
■ ■ ■■■ MyApp.Api.csproj
■ ■ (references: Application,
■ ■ Infrastructure)
■■■ tests/
■■■ MyApp.Domain.Tests/
■■■ MyApp.Application.Tests/
■■■ MyApp.Integration.Tests/

```

Key Takeaways for 17.1

- Domain -- pure C# entities and business rules; zero dependencies
- Application -- use cases and service interfaces; depends on Domain only
- Infrastructure -- EF Core, email, external APIs; implements Application interfaces
- API -- controllers and HTTP; depends on Application (not Infrastructure directly)
- Dependencies only point inward -- outer knows inner, inner never knows outer
- Business rules live in Domain -- not in controllers or repositories

Layer	References
Domain	nothing
Application	Domain
Infrastructure	Application + Domain
API	Application + Domain (Infra only for DI wiring)

Memory aid: Layered architecture is a wedding cake. The bottom tier (Domain) supports everything above but knows nothing about upper tiers. Each tier rests on the one below. The top tier (API) is decorative -- but collapses without the layers beneath it.

Debugging Tips

Symptom	Cause	Fix
Application project references EF Core	Infrastructure concern leaking into Application	Move EF Core to Infrastructure; Application only has interfaces

Symptom	Cause	Fix
Controller directly uses ApplicationDbContext	Dependency rule violation	Inject IOrderRepository interface; let DI resolve SqlOrderRepository
Domain entity has [Required] data annotation	EF Core concern leaking into Domain	Use Fluent API in Infrastructure config; keep Domain classes pure
Cannot test service without a real database	Service directly uses ApplicationDbContext	Inject IOrderRepository; test with InMemoryOrderRepository

17.2. Separation of Concerns Across Layers

Builds on: 17.1 (layered architecture -- layers defined there), 9.1 (SOLID -- SRP is the principle behind SoC), 9.5 (DI -- mechanism that keeps layers decoupled).

What belongs where

Domain Layer:

- Entity classes (Order, Customer, Product)
- Value objects (Address, Money, DateRange)
- Business rule methods (order.Cancel())
- Domain exceptions (DomainException)
- Domain events (OrderCreatedEvent)
- NOT: EF Core attributes, HTTP concepts, logging

Application Layer:

- Use case orchestration (OrderService.CreateAsync)
- Repository interfaces (IOrderRepository)
- External service interfaces (IEmailService)
- Commands/Queries and DTOs
- Validators (FluentValidation)
- Mapping (entity to DTO and back)
- NOT: EF Core, SQL, HttpContext, SMTP details

Infrastructure Layer:

- EF Core DbContext and configurations
- Repository implementations
- Email, external API clients, file storage
- NOT: business logic, HTTP, controller code

API Layer:

- Controllers and Minimal API endpoints
- Middleware, action filters
- Program.cs -- DI registration, pipeline
- Auth and authorisation setup
- NOT: business logic, direct DB access

Wrong -- business logic in a controller

```
// WRONG -- controller doing too much:
[HttpPost]
public async Task<IActionResult> CreateOrder(
    CreateOrderDto dto)
```

```

{
    // Validation logic in controller:
    if (dto.Amount <= 0)
        return BadRequest("Amount invalid");

    // Business rule in controller:
    var customer = await _context.Customers
        .FindAsync(dto.CustomerId);
    if (customer!.CreditLimit < dto.Amount)
        return BadRequest(
            "Exceeds credit limit");

    // Data access in controller:
    var order = new Order { ... };
    _context.Orders.Add(order);
    await _context.SaveChangesAsync();

    // Email in controller:
    await _emailClient.SendAsync(
        customer.Email, "Confirmed", $"...");

    return CreatedAtAction(
        nameof(GetById), new { id = order.Id },
        order);
}

```

Correct -- each concern in its layer

```

// Controller only: receive, delegate, respond
[HttpPost]
public async Task<IActionResult> Create(
    CreateOrderDto dto, CancellationToken ct)
{
    var order = await _orderService
        .CreateAsync(dto.ToCommand(), ct);
    return CreatedAtAction(
        nameof(GetById),
        new { id = order.Id },
        order.ToDto());
}

// Application -- orchestrates use case:
public async Task<Order> CreateAsync(
    CreateOrderCommand cmd, CancellationToken ct)
{
    var customer = await _customerRepo
        .GetByIdAsync(cmd.CustomerId, ct)
        ?? throw new NotFoundException(
            "Customer not found");

    if (customer.CreditLimit < cmd.Amount)
        throw new BusinessException(
            "Amount exceeds credit limit");

    var order = new Order();
}

```



```

        foreach (var item in cmd.Items)
            order.AddItem(item.ToEntity());

        var saved = await _repo
            .CreateAsync(order, ct);
        await _email
            .SendOrderConfirmationAsync(saved, ct);
        return saved;
    }

    // Domain -- business rules on entity:
    public void AddItem(OrderItem item)
    {
        if (Status != OrderStatus.Pending)
            throw new DomainException(
                "Cannot add to non-pending order");
        _items.Add(item);
    }

    // Infrastructure -- data access only:
    public async Task<Order> CreateAsync(
        Order order, CancellationToken ct)
    {
        _context.Orders.Add(order);
        await _context.SaveChangesAsync(ct);
        return order;
    }

```

Mapping -- where it belongs

```

// Application layer -- mapping extension methods:
public static class OrderMappings
{
    // DTO -> Command (Application input):
    public static CreateOrderCommand ToCommand(
        this CreateOrderDto dto)
        => new(
            dto.CustomerId,
            dto.Items.Select(i =>
                new OrderItemCommand(
                    i.ProductId,
                    i.Quantity,
                    i.UnitPrice)).ToList());

    // Domain Entity -> DTO (API response):
    public static OrderDto ToDto(
        this Order order)
        => new(
            order.Id,
            order.CustomerName,
            order.Status.ToString(),
            order.Amount,
            order.CreatedAt,
            order.Items
                .Select(i => i.ToDto())

```

```

        .ToList());
    }

```

Exception handling separation

```

// Domain layer -- domain exceptions:
public class DomainException : Exception
{
    public DomainException(string message)
        : base(message) { }
}

// Application layer -- use-case exceptions:
public class NotFoundException : Exception
{
    public NotFoundException(string msg)
        : base(msg) { }
}

public class BusinessException : Exception
{
    public BusinessException(string msg)
        : base(msg) { }
}

// API layer -- handle all in one place:
public class GlobalExceptionMiddleware
{
    public async Task InvokeAsync(HttpContext ctx)
    {
        try { await _next(ctx); }
        catch (NotFoundException ex)
        {
            await WriteProblem(ctx, 404, ex.Message);
        }
        catch (DomainException ex)
        {
            await WriteProblem(ctx, 422, ex.Message);
        }
        catch (BusinessRuleException ex)
        {
            await WriteProblem(ctx, 400, ex.Message);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Unhandled");
            await WriteProblem(ctx, 500,
                "An unexpected error occurred");
        }
    }
}

```

Validation separation -- two distinct layers

```

// Layer 1 -- HTTP format validation:
public class CreateOrderDtoValidator

```

```

        : AbstractValidator<CreateOrderDto>
    {
        public CreateOrderDtoValidator()
        {
            RuleFor(x => x.CustomerId)
                .GreaterThan(0);
            RuleFor(x => x.Amount)
                .GreaterThan(0)
                .LessThanOrEqualTo(1_000_000);
            RuleFor(x => x.Items).NotEmpty();
        }
    }

    // Layer 2 -- Business rule validation in service:
    if (customer.IsBlacklisted)
        throw new BusinessException(
            "Blacklisted customers cannot order");

    if (customer.CreditLimit < cmd.Amount)
        throw new BusinessException(
            $"Exceeds credit limit of "
            + $"{customer.CreditLimit:C}");

```

Key Takeaways for 17.2

- SoC = each class and layer has one clearly defined responsibility
- Domain: entities and business rules -- no HTTP, no EF Core
- Application: orchestrates use cases via interfaces -- no EF Core or HTTP
- Infrastructure: implements interfaces -- no business logic
- API: receives HTTP, delegates, returns response -- no business logic
- Exception types: defined in originating layer; handled centrally in API middleware
- Validation: two layers -- HTTP format (API) + business rules (Application/Domain)

Warning signs of wrong-layer code:

```

Controller calls _context directly
Service imports EF Core namespace
Domain entity has [Column] attribute
Repository throws HttpException

```

Memory aid: SoC is a kitchen brigade. The waiter (API) takes orders. The sous-chef (Application) coordinates. The line cooks (Infrastructure) handle specific stations. The head chef (Domain) defines what makes the dish correct. No station does another's job.

Debugging Tips

Symptom	Cause	Fix
Changing DB schema breaks controllers	Controllers use EF Core entities as DTOs directly	Introduce separate DTOs; map entity to DTO in Application layer
Cannot unit test service without a real database	Service directly uses AppDbContext	Inject IOrderRepository; test with InMemoryOrderRepository

Symptom	Cause	Fix
Validation logic duplicated across layers	Validation scattered across controller, service, entity	Format validation in one validator; business rules in domain/service
Exception handling repeated in every controller	No centralised exception handling	Implement GlobalExceptionHandler; catch once centrally

17.3. Repository and Unit of Work Pattern in Practice

Builds on: 9.2 (repository pattern -- the concept), 17.1 (layered architecture -- repositories live in Infrastructure), 14.8 (transactions -- Unit of Work coordinates them).

The problem Unit of Work solves

```
// WITHOUT Unit of Work -- partial saves possible:
var order = await _orderRepo
    .CreateAsync(cmd.ToEntity(), ct);
// First save committed OK

await _inventoryRepo
    .ReserveStockAsync(cmd.Items, ct);
// If this throws: order exists but stock
// not reserved -- inconsistent state!

// WITH Unit of Work -- all or nothing:
_orderRepo.Add(cmd.ToEntity());
// No save yet

_inventoryRepo.ReserveStock(cmd.Items);
// No save yet

await _uow.SaveChangesAsync(ct);
// All committed in ONE transaction
// Exception -> nothing saved
```

IUnitOfWork interface

```
// Defined in Application layer:
public interface IUnitOfWork
{
    Task<int> SaveChangesAsync(
        CancellationToken ct = default);
    Task BeginTransactionAsync(
        CancellationToken ct = default);
    Task CommitTransactionAsync(
        CancellationToken ct = default);
    Task RollbackTransactionAsync(
        CancellationToken ct = default);
}
```

Unit of Work implementation

```
// Infrastructure layer:
```

```

public class UnitOfWork : IUnitOfWork
{
    private readonly AppDbContext _context;
    private IDbContextTransaction? _tx;

    public UnitOfWork(AppDbContext context)
        => _context = context;

    public async Task<int> SaveChangesAsync(
        CancellationToken ct = default)
        => await _context.SaveChangesAsync(ct);

    public async Task BeginTransactionAsync(
        CancellationToken ct = default)
        => _tx = await _context.Database
            .BeginTransactionAsync(ct);

    public async Task CommitTransactionAsync(
        CancellationToken ct = default)
    {
        if (_tx is null)
            throw new InvalidOperationException(
                "No active transaction");
        await _tx.CommitAsync(ct);
        await _tx.DisposeAsync();
        _tx = null;
    }

    public async Task RollbackTransactionAsync(
        CancellationToken ct = default)
    {
        if (_tx is null) return;
        await _tx.RollbackAsync(ct);
        await _tx.DisposeAsync();
        _tx = null;
    }
}

```

Repository implementations

```

// Application layer -- interface:
public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(
        int id, CancellationToken ct);
    Task<List<Order>> GetByUserAsync(
        int userId, CancellationToken ct);
    Order Add(Order order);
    void Update(Order order);
    void Remove(Order order);
}

// Infrastructure layer -- EF Core impl:
public class SqlOrderRepository : IOrderRepository
{

```

```

private readonly AppDbContext _context;

public SqlOrderRepository(AppDbContext ctx)
    => _context = ctx;

// Reads -- materialise immediately:
public async Task<Order?> GetByIdAsync(
    int id, CancellationToken ct)
    => await _context.Orders
        .AsNoTracking()
        .Include(o => o.Items)
        .FirstOrDefaultAsync(
            o => o.Id == id, ct);

public async Task<List<Order>> GetByUserAsync(
    int userId, CancellationToken ct)
    => await _context.Orders
        .AsNoTracking()
        .Where(o => o.UserId == userId)
        .OrderByDescending(o => o.CreatedAt)
        .ToListAsync(ct);

// Writes -- no save, return entity:
public Order Add(Order order)
{
    _context.Orders.Add(order);
    return order;
    // SaveChanges deferred to IUnitOfWork
}

public void Update(Order order)
    => _context.Orders.Update(order);

public void Remove(Order order)
    => _context.Orders.Remove(order);
}

```

DI registration

```

builder.Services.AddDbContext<AppDbContext>(
    o => o.UseSqlServer(
        builder.Configuration
            .GetConnectionString("Default")));

builder.Services
    .AddScoped<IUnitOfWork, UnitOfWork>();

builder.Services
    .AddScoped<IOrderRepository,
        SqlOrderRepository>();
builder.Services
    .AddScoped<ICustomerRepository,
        SqlCustomerRepository>();
builder.Services
    .AddScoped<IInventoryRepository,

```

```

        SqlInventoryRepository>());

builder.Services.AddScoped<OrderService>();

```

Complete service using Unit of Work

```

public class OrderService
{
    private readonly IOrderRepository _orders;
    private readonly ICustomerRepository _customers;
    private readonly IInventoryRepository _inventory;
    private readonly IEmailService _email;
    private readonly IUnitOfWork _uow;

    public async Task<Order> PlaceOrderAsync(
        CreateOrderCommand cmd, CancellationToken ct)
    {
        var customer = await _customers
            .GetByIdAsync(cmd.CustomerId, ct)
            ?? throw new NotFoundException(
                "Customer not found");

        foreach (var item in cmd.Items)
        {
            var reserved = await _inventory
                .ReserveAsync(
                    item.ProductId,
                    item.Quantity, ct);
            if (!reserved)
                throw new BusinessException(
                    $"Insufficient stock for "
                    + $"product {item.ProductId}");
        }

        var order = new Order
        {
            CustomerId = cmd.CustomerId,
            CustomerName = customer.Name,
            Status = OrderStatus.Pending,
            CreatedAt = DateTime.UtcNow
        };
        foreach (var item in cmd.Items)
            order.AddItem(new OrderItem(
                item.ProductId,
                item.Quantity,
                item.UnitPrice));

        // Add to context -- no save yet:
        _orders.Add(order);

        // Commit all changes atomically:
        await _uow.SaveChangesAsync(ct);

        // Email OUTSIDE transaction:
        try

```

```

        {
            await _email
                .SendOrderConfirmationAsync(
                    order, ct);
        }
        catch (Exception ex)
        {
            _logger.LogError(ex,
                "Confirmation email failed "
                + "for order {Id}", order.Id);
        }

        return order;
    }
}

```

When to use each transaction approach

Single SaveChangesAsync (implicit transaction):

- One operation touching one aggregate
- Creating Order + Items in one call
- Default choice for most operations

IUnitOfWork.SaveChangesAsync:

- Multiple repos that must be atomic
- Order + inventory + payment together
- Cross-aggregate operations

Explicit BeginTransactionAsync:

- Multiple SaveChangesAsync calls must be atomic
- Need savepoints for partial rollback

Avoid the thin generic repository anti-pattern

// ANTI-PATTERN -- thin generic wrapping EF Core:

```

public interface IRepository<T>
{
    IQueryable<T> GetAll(); // leaks EF Core!
    T GetById(int id);
    void Add(T entity);
}
// IQueryable leaks EF Core outside Infrastructure
// Callers build LINQ -- bypasses the layer

```

// CORRECT -- domain-specific repository:

```

public interface IOrderRepository
{
    Task<Order?> GetByIdAsync(
        int id, CancellationToken ct);
    Task<List<Order>> GetPendingAsync(
        CancellationToken ct);
    Task<PagedResult<Order>> SearchAsync(
        OrderSearchParams p, CancellationToken ct);
    Order Add(Order order);
    void Update(Order order);
}

```



```

}
// Expresses domain intent -- not EF Core ops
// Swappable -- could implement with Dapper

```

Key Takeaways for 17.3

- Unit of Work controls when SaveChangesAsync is called -- defers saves for atomicity
- All repos sharing the same Scoped DbContext share the same Unit of Work
- Repo write methods (Add, Update, Remove) modify change tracker but don't save
- IUnitOfWork.SaveChangesAsync commits all pending changes across all repos at once
- External calls (email, events) belong OUTSIDE the transaction -- log, don't throw
- Avoid IQueryable<T> in repo interfaces -- leaks EF Core outside Infrastructure
- Domain-specific repo methods express intent, not EF Core operations

```

Service calls _orders.Add(order)      -> tracked, not saved
Service calls _inventory.Reserve()    -> tracked, not saved
Service calls _payment.Record()       -> tracked, not saved
Service calls _uow.SaveChangesAsync() -> ALL saved atomically

```

Memory aid: The Unit of Work is a shopping trolley. Repositories are shelves -- you pick items from each (Add to context). No money changes hands yet. The Unit of Work is the checkout till -- everything paid for in one transaction. If the card declines (exception), nothing is taken from any shelf.

Debugging Tips

Symptom	Cause	Fix
Order saved but stock not reserved after exception	Each repo calls its own SaveChangesAsync()	Move saves to IUnitOfWork.SaveChangesAsync() at end of service method
IQueryable executes outside repository	Repository returns IQueryable instead of materialised list	Change return type to List<T>; call ToListAsync() inside the repo
Repositories not sharing the same transaction	Each service creates its own DbContext instance	Ensure DbContext is Scoped; all repos inject the same instance per request
Email sent even when order save failed	Email called inside transaction before commit	Move email after SaveChangesAsync(); wrap in try/catch

Section 17 -- Complete Key Takeaways

Topic	Core concept	Key pattern
17.1 Layered Arch	Dependencies inward only	4 projects: Domain, App, Infra, API
17.2 Separation	One responsibility per class/layer	Interface in App, impl in Infra
17.3 Repo + UoW	Deferred saves for atomicity	_repo.Add() then _uow.SaveChanges()

Tier 1 Roadmap Complete

You have now covered all Tier 1 topics from the roadmap -- Sections 0 through 17. This represents the full core competency for a Fullstack .NET Developer role: C# fundamentals, OOP, LINQ, async, design patterns, testing, Git, ASP.NET Core Web API, Entity Framework Core, authentication, and Clean Architecture.

Tier 2 topics (Understand conceptually, go deep only when a project needs it): Caching (IMemoryCache, Redis), SignalR, BackgroundService, Polly resilience, CQRS/MediatR, Reflection, File I/O, Singleton/Decorator/Observer patterns, and deployment basics.